LGTM: Technical Supporting Material

Design diagrams, tool and prompt definitions, judge details and the external technologies behind LGTM. Matches the final submission (main, 10 Nov 2026); results are from the final evaluation run at commit af878ce.

1. Overall design

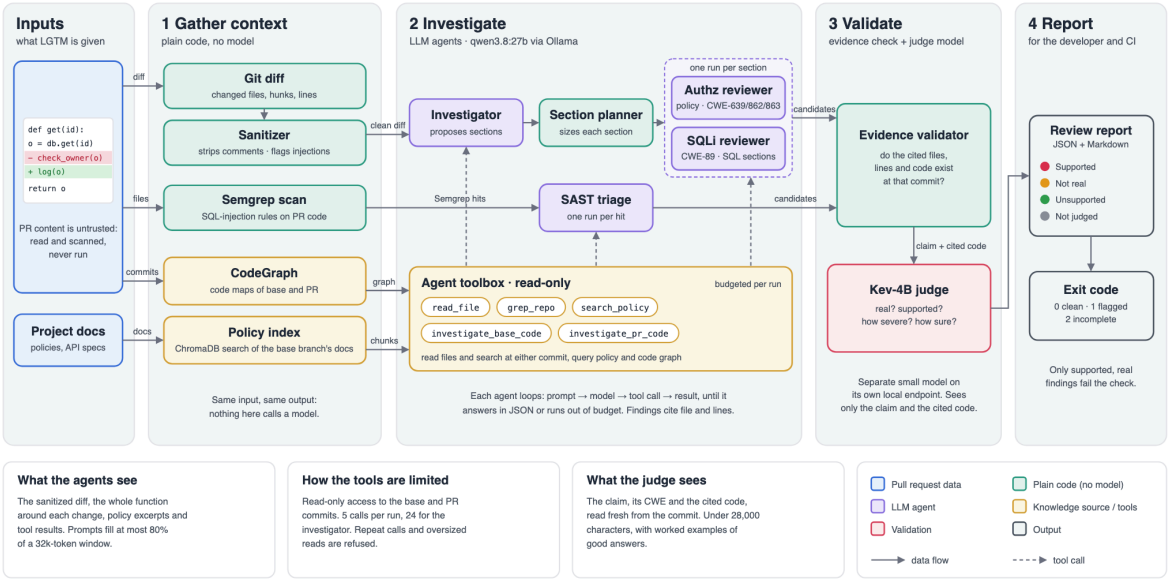


Figure 1. Plain code gathers context, LLM agents propose candidates, an evidence validator and a separate judge decide which stand, and the report sets the exit code. The authz and SQLi reviewers both run on each section; the SQLi reviewer only where the code builds SQL. Source: docs/architecture.png.

2. Tool definitions

Five read-only tools over committed snapshots; `base` and `pr` are each resolved once to a commit SHA, so no read depends on the working tree. Budgets are 5 calls per run (24 for the investigator). The graph tools go only to the investigator, the authz reviewer and the SQLi reviewer, and only when a graph session exists.

Tool	Arguments	Backend	Limits
`read_file`	`path, start_line, end_line, snapshot`	`git cat-file blob <sha>:<path>`	600 lines, 8,000 chars; the investigator may not read a whole changed file
`grep_repo`	`pattern, file_glob, snapshot`	`git grep -n -E` at the commit	30 matches; single-line valid regex
`search_policy`	`query`	ChromaDB `lgtm_docs`, top 5 chunks	8,000 chars
`investigate_base_code`	`query`	CodeGraph index of the base snapshot	6,000 chars, 20 s
`investigate_pr_code`	`query`	CodeGraph index of the PR snapshot	6,000 chars, 20 s

3. Prompt contracts

All model-facing text lives in `src/lgtm/agents/prompts.toml`, read through a registry, and every PR-derived block is labelled untrusted.

Agent	Contract	Output
`investigator`	Group changed hunks into sections, one access-control concern each, by manifest hunk id; name outside code each depends on	`sections[]` with `hunks[]`, `context[]`
`authz_reviewer`	Compare a section with the documented policy for CWE-639, 862, 863. Report only changed or removed lines, or code the PR newly exposes. No explicit rule is not a violation	`findings[]` with `file`, `line`, possibly empty
`sqli_reviewer`	Review a section for CWE-89 without waiting for Semgrep: SQL built from attacker-influenced values by concatenation, f-strings, % or .format and then executed. Bound parameters, binding ORM filters and allowlisted identifiers are not candidates. Runs only on sections that show SQL	`findings[]` with `file`, `line`, possibly empty
`sast_triage`	Triage one Semgrep CWE-89 finding. Likely false positive if bound parameters, an allowlist, or no user input reaches the query	`finding_id`, `cwe_id`, `explanation`, `evidence[]`, `suggested_fix`
`plain_reviewer`	Baseline: diff only, no tools; classify `sqli` and `authz` with a verdict	`assessments[]` with `confidence`

4. Evidence validation and judge

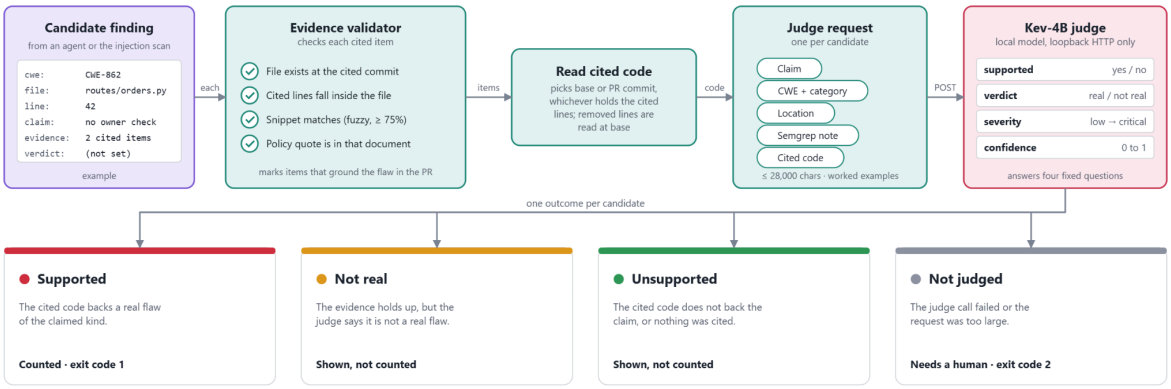


Figure 2. Agents only propose candidates. The validator checks the citations exist; Kev-4B decides whether the cited code backs the claim. Source: docs/validation.png.

Evidence validator: the file exists at the cited commit; line ranges are ordered and inside the file; the quoted snippet matches at 0.75 fuzzy similarity (elision lines allowed); a policy quote appears in its document. **Judge:** four fixed questions per candidate (`supported`, `verdict`, `severity`, `provenance_confidence`) over loopback HTTP only, with no proxy or redirects, a 1 MB reply cap, a 60 s timeout and a 28,000-character request cap. A candidate with no readable cited source is unsupported; a failed or oversized request leaves it not judged (exit code 2).

5. External technologies

Technology	Role	Why it was chosen	Limit

Semgrep CE	Four bundled CWE-89 rules run on the PR files offline, `--metrics off`	Deterministic, high-recall SQLi signal for the LLM to triage; short auditable rules; no code leaves the machine	Alone it caught 3 of 17 held-out advisories, with 2 false alarms on their fixes; blind to authorization; Python and CWE-89 only
CodeGraph optional	`investigate_*` tools: definitions, callers and call paths per snapshot	Authorization flaws depend on code outside the diff, such as the decorator guarding a route; one query answers it within the call budget	Needs Node and index time; without it agents fall back to `read_file` and `grep_repo`
ChromaDB + `all-MiniLM-L6-v2`	Indexes the repo's policy and API docs, top 5 chunks per query	A flaw is only a flaw against a documented rule; embedded, no server, CPU only	Re-index after doc changes; can miss differently worded rules
Ollama + `qwen3.8:27b`	Default reviewer for every agent, local or on a GPU cluster	Code stays on controlled machines; native tool calls. The 7B `qwen2.5-coder` never used its tools (0 calls in 124 findings)	Needs a 24 GB GPU; Q4_K_M quantised
Amazon Bedrock (optional)	Cloud reviewer for the model comparison (Claude Sonnet 4.6, Claude Haiku 4.5, gpt-oss-120b); the demo's default reviewer when no GPU is set up	Tests whether a bigger model helps, and lets anyone run LGTM without a GPU	Code leaves the machine; 3 to 11 false alarms per run against Qwen's 1; costs money, so runs stop at a set budget
Kev-4B	Judge that sees only the claim and the cited code	An independent check the reviewer cannot talk around; structured answers	Strict: in the component benchmark it rejected the one real django finding (verdicts right on 25 of 28; labels provisional); fails closed
LangChain `0.3`	`ChatOllama` client and tool schemas; loop and validation are LGTM's own	Native tool-calling without hand-written HTTP	Pinned below 0.4